

Kubernetes – Prérequis & Fondamentaux

Jour 3 — Fondamentaux Kubernetes

Partie 2 : configuration, stockage, exposition externe

Kubernetes — Prérequis & Fondamentaux — m2i
Module 3/3

Objectifs du module

À la fin de cette journée, vous serez capable de :

- Externaliser la configuration d'une application via ConfigMap et Secret
- Gérer du stockage persistant via PV, PVC et StorageClass
- Choisir entre Deployment, StatefulSet, DaemonSet, Job et CronJob
- Exposer une application à l'extérieur du cluster via un Ingress
- Composer un Pod multi-conteneurs (sidecar) et le diagnostiquer
- Suivre une méthode de debug structurée

Durée : 7 heures (1 journée)

Plan

1. ConfigMaps et Secrets
2. Stockage persistant
3. StatefulSets, DaemonSets, Jobs
4. Ingress et exposition externe
5. Patterns multi-conteneurs et observabilité
6. Synthèse et bonnes pratiques
7. Démonos et TP

1. ConfigMaps et Secrets

Découpler code et configuration

“ Une bonne application ne contient pas sa config : elle la reçoit à l'exécution. ”
Bénéfices :

- Une seule image, plusieurs environnements
- Rotation de secrets sans rebuild
- Suivi des changements de config indépendant du code
- Sécurité : pas de credentials dans le repo Docker

C'est aussi le principe 3 du 12-factor app.

ConfigMap : création

```
# À partir de fichiers
kubectl create configmap app-config \
  --from-file=application.yaml \
  --from-literal=LOG_LEVEL=INFO

# Manifeste YAML équivalent
```

```
apiVersion: v1
kind: ConfigMap
metadata: {name: app-config}
data:
  LOG_LEVEL: "INFO"
  application.yaml: |
    server:
      port: 8080
    spring:
      profiles:
        active: prod
```

Consommation d'une ConfigMap

```
spec:
  containers:
    - name: app
      image: app:1.0
      envFrom:                                # tout en variables
        - configMapRef: {name: app-config}
      env:                                    # variables individuelles
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: LOG_LEVEL
      volumeMounts:                           # ou comme fichier
        - name: cfg
          mountPath: /etc/app
  volumes:
    - name: cfg
      configMap: {name: app-config}
```


Secrets : types et création

Type	Usage
Opaque	Données libres (mot de passe, token)
kubernetes.io/tls	Certificat + clé pour Ingress
kubernetes.io/dockerconfigjson	Pull d'image depuis registry privé
kubernetes.io/service-account-token	Auto-géré par K8s

```
kubectl create secret generic db-creds \  
  --from-literal=DB_USER=app \  
  --from-literal=DB_PASSWORD='S3cret!'  
  
kubectl create secret tls web-tls \  
  --cert=fullchain.pem --key=privkey.pem
```

Secrets : sécurité réelle

“ Les Secrets sont stockés en clair dans etcd par défaut. ”

Mesures recommandées :

- Activer l'encryption at rest d'etcd (`EncryptionConfiguration`)
- RBAC strict sur la ressource Secret
- Externaliser via External Secrets Operator + Vault ou AWS Secrets Manager
- Auditer les accès via les logs apiserver

L'encodage base64 n'est PAS un chiffrement.

Mise à jour de configuration

```
kubectl edit configmap app-config  
# ou  
kubectl apply -f configmap.yaml
```

Propagation :

- ConfigMap monté en volume : mise à jour dans le Pod après ~60s (kubelet sync)
- ConfigMap consommée en `env` : pas de propagation, redémarrage requis
- Pattern Reloader : annotation déclenche un rollout automatique

```
metadata:  
  annotations:  
    reloader.stakater.com/auto: "true"
```

2. Stockage persistant

Volumes éphémères

Vivent et meurent avec le Pod :

```
spec:
  containers:
    - name: app
      volumeMounts:
        - {name: cache, mountPath: /cache}
        - {name: cfg, mountPath: /etc/app}
  volumes:
    - name: cache
      emptyDir: {} # tmpfs partagé entre conteneurs
    - name: cfg
      configMap: {name: app-config}
    - name: secret
      secret: {secretName: db-creds}
    - name: meta
      downwardAPI: # métadonnées du Pod
        items: [{path: name, fieldRef: {fieldPath: metadata.name}}]
```

PV et PVC : découplage

```
+-- Cluster -----+
|
| StorageClass (template provisioning)
|   |
|   v
| PersistentVolume (ressource cluster)
|   ^
|   | bound to
|   |
| PersistentVolumeClaim (demande)
|   ^
|   | utilisé par
|   Pod
+-----+
```

Le PVC est la demande, le PV la fourniture. Le binding est géré par K8s.

Exemple PVC + Pod

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard
---
apiVersion: v1
kind: Pod
metadata: {name: db}
spec:
  containers:
    - name: postgres
      image: postgres:16
      volumeMounts: [{name: data, mountPath: /var/lib/postgresql/data}]
  volumes:
    - name: data
      persistentVolumeClaim: {claimName: data}
```

StorageClass et provisioning dynamique

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: fast}
provisioner: rancher.io/local-path
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

- **provisioner** : qui crée le PV (cloud, local-path, Ceph, etc.)
- **reclaimPolicy** : Delete ou Retain quand le PVC disparaît
- **volumeBindingMode** : WaitForFirstConsumer aligne le PV au nœud du Pod

Sur Kind : local-path-provisioner fait le travail par défaut.

Access modes et reclaim policies

Access mode	Sigle	Cas d'usage
ReadWriteOnce	RWO	Base de données mono-écrivain
ReadOnlyMany	ROX	Fichiers statiques partagés
ReadWriteMany	RWX	Stockage partagé NFS/CephFS
ReadWriteOncePod	RWOP	RWO restreint à 1 Pod (K8s 1.27+)

```
kubectl get pv,pvc -A  
kubectl describe pvc data
```

3. StatefulSets, DaemonSets, Jobs

StatefulSet : identités stables

“ Un Deployment traite ses Pods comme du bétail. Un StatefulSet leur donne un nom et un domicile. ”

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: db}
spec:
  serviceName: db                # headless Service obligatoire
  replicas: 3
  selector: {matchLabels: {app: db}}
  template:
    metadata: {labels: {app: db}}
    spec:
      containers: [{name: postgres, image: postgres:16}]
  volumeClaimTemplates:          # un PVC par réplique
    - metadata: {name: data}
      spec:
        accessModes: [ReadWriteOnce]
        resources: {requests: {storage: 5Gi}}
```

Pods : db-0, db-1, db-2. Chacun garde son PVC à travers les reschedules.

DaemonSet : un Pod par nœud

```
apiVersion: apps/v1
kind: DaemonSet
metadata: {name: node-exporter, namespace: monitoring}
spec:
  selector: {matchLabels: {app: node-exporter}}
  template:
    metadata: {labels: {app: node-exporter}}
    spec:
      hostNetwork: true
      containers:
        - name: node-exporter
          image: prom/node-exporter:v1.8.0
          ports: [{containerPort: 9100, hostPort: 9100}]
```

Cas d'usage : agents système (logs Fluent Bit, métriques node-exporter, CNI, kube-proxy).

Jobs et CronJobs

```
apiVersion: batch/v1
kind: Job
metadata: {name: migrate-db}
spec:
  backoffLimit: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: migrate
          image: app-migrator:1.0
---
apiVersion: batch/v1
kind: CronJob
metadata: {name: backup}
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers: [{name: backup, image: backup:1.0}]
```

4. Ingress et exposition externe

Pourquoi Ingress ?

Limites de NodePort

- Plage 30000-32767
- Un port par service exposé
- Difficile à mémoriser

L'Ingress résout les 3 : un point d'entrée, routage par hostname/path, TLS unifié.

Limites de LoadBalancer

- 1 IP par service = coût
- Pas de routage par hostname
- Pas de TLS centralisé

Composants

```
+--- Internet ----+
      |
      v
+-- LoadBalancer / NodePort --+
      |
      v
+-- Ingress Controller -----+ (Pod géré comme un Deployment)
      | lit les règles
      v
+-- Ingress (règles) -----+
      |
      v
+-- Services internes -----+
      |
      v
+-- Pods -----+
```

L'Ingress (l'objet K8s) est inerte sans Ingress Controller (l'agent qui l'applique).

Règles d'Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
    - hosts: [app.m2i.local]
      secretName: app-tls
  rules:
    - host: app.m2i.local
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service: {name: api, port: {number: 8080}}
```

Installer ingress-nginx sur Kind

```
kubectl apply -f https://raw.githubusercontent.com/\
kubernetes/ingress-nginx/controller-v1.11.2/deploy/static/provider/kind/deploy.yaml

kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=120s

# Sur Kind, ajouter "extraPortMappings" 80 et 443 dans la config du cluster
```

Le cluster Kind doit avoir mappé les ports 80/443 sur l'hôte (config `kind`).

Gateway API : la suite

“ Gateway API est l'évolution officielle de l'Ingress, plus expressive et orientée multi-équipes. ”

Concept	Ingress	Gateway API
Controller config	Annotations	GatewayClass
Routes	Mélangées	HTTPRoute, TCPRoute, etc.
Multi-équipes	Difficile	Séparation contrôleur / route
Statut	Stable mais figé	Beta/GA selon ressources

Pour les nouveaux projets : tester Gateway API.

5. Patterns multi-conteneurs et observabilité

Le pattern sidecar

“ Un sidecar étend les capacités du conteneur principal sans modifier son code. ”

Cas d'usage classiques :

- **Proxy** : Envoy ou Istio pour TLS et observability
- **Logging** : Fluent Bit qui pousse les logs vers Loki/Elastic
- **Adapter** : convertit un format de sortie maison vers Prometheus
- **Sync** : git-sync qui rafraîchit un volume depuis un repo

Tous les conteneurs du Pod sont co-schedulés et co-démarrent.

Pod sidecar : exemple Java + Python

```
spec:
  containers:
    - name: api
      image: spring-app:1.0
      volumeMounts: [{name: logs, mountPath: /var/log/app}]
    - name: log-shipper
      image: python-log-shipper:1.0
      env: [{name: LOG_DIR, value: /var/log/app}]
      volumeMounts: [{name: logs, mountPath: /var/log/app, readOnly: true}]
  volumes:
    - name: logs
      emptyDir: {}
```

Communication : volume `emptyDir` partagé. Aucun écouteur réseau ouvert.

Consultation des logs

```
kubectl logs api-abc123                # 1er conteneur par défaut
kubectl logs api-abc123 -c log-shipper  # conteneur précis
kubectl logs -f api-abc123 --since=5m   # suivi 5 dernières min
kubectl logs api-abc123 --previous      # logs du crash précédent
kubectl logs -l app=api --tail=50 --max-log-requests=10
```

`--previous` est crucial pour comprendre pourquoi un Pod CrashLoop.

Métriques de base

```
kubectl top nodes  
kubectl top pods -A --sort-by=memory  
kubectl top pods -n prod --containers
```

Prérequis : `metrics-server` déployé dans le cluster.

```
kubectl apply -f https://github.com/kubernetes-sigs/\  
metrics-server/releases/latest/download/components.yaml
```

Sur Kind : ajouter `--kubelet-insecure-tls` dans l'argument du Deployment.

Méthode de debug

Sept étapes pour un Pod qui ne démarre pas :

1. `kubectl get pod <name>` — état général
2. `kubectl describe pod <name>` — section Events en bas
3. `kubectl logs <name> -c <container>` — sortie applicative
4. `kubectl logs <name> --previous` — logs du crash précédent
5. `kubectl get events --sort-by=.lastTimestamp` — événements du namespace
6. `kubectl exec -it <name> -- sh` — investigation à chaud
7. Si Pending : `kubectl describe node` — capacité, taints

Le `describe pod` répond à 80% des cas.

6. Synthèse et bonnes pratiques

Organisation des manifestes

```
deploy/  
  base/  
    deployment.yaml  
    service.yaml  
    configmap.yaml  
    kustomization.yaml  
  overlays/  
    dev/  
      kustomization.yaml  
      replicas-patch.yaml  
    prod/  
      kustomization.yaml  
      replicas-patch.yaml  
      secret-patch.yaml
```

Une seule source de vérité par environnement, surcouches localisées.

Conventions de nommage et labels

```
metadata:  
  name: web  
  labels:  
    app.kubernetes.io/name: web  
    app.kubernetes.io/instance: web-prod  
    app.kubernetes.io/version: "1.2.3"  
    app.kubernetes.io/component: api  
    app.kubernetes.io/part-of: monitoring  
    app.kubernetes.io/managed-by: helm
```

Les labels `app.kubernetes.io/*` sont le standard recommandé. Ils permettent les filtres `kubectl get pod -l app.kubernetes.io/name=web`.

Namespaces et isolation

Stratégie	Pour quoi
1 namespace = 1 environnement	Petits projets
1 namespace = 1 équipe	Multi-tenant léger
1 namespace = 1 application	Forte isolation + RBAC
kube-system, default	Réservés / défaut, ne pas y déployer

Toujours associer `NetworkPolicy` + `ResourceQuota` à un namespace de prod.

Préparation au Module 2 de la formation

Ce que nous verrons dans les 2 jours suivants (parcours complet) :

- Helm : packager les manifestes en charts paramétrables
- ArgoCD : déploiement continu déclaratif (GitOps)
- Pipeline complet : Git → image → ArgoCD → cluster
- Stratégies blue/green et canary avec Argo Rollouts

Les fondations de ces 3 jours conditionnent la suite.

